

ArcFS: A Content-Addressable FUSE Filesystem with Inline Deduplication, Copy-on-Write Versioning, and Semantic Tag Navigation

Anandkumar N S, Dhakkshin S R, Kishoreadhith V, M Raj Ragavender, Rithvik K

Department of Computer Science and Engineering

PSG College of Technology, Anna University

Coimbatore, India – 641 004

{22z209, 22z215, 22z232, 22z233, 22z253}@psgtech.ac.in

Dr. Arul Anand N (Faculty Guide)

Department of Computer Science and Engineering

PSG College of Technology

Coimbatore, India – 641 004

Abstract—Traditional filesystems store data by location, which prevents native deduplication, makes versioning expensive, and confines every file to a single position in the directory hierarchy. We present ArcFS, a user-space filesystem written in Rust that integrates three storage optimizations into a single transparent write path: (i) content-defined chunking (CDC) via a FastCDC Gear-hash engine with 2 KB to 64 KB variable chunk sizes, (ii) SHA-256 content-addressable storage (CAS) with Zstandard level-3 compression, and (iii) copy-on-write (CoW) snapshots at $O(1)$ metadata cost. A tag-based virtual filesystem (TagFS) overlays the namespace and resolves virtual directory paths through lazy set-intersection, allowing files to appear under multiple tag directories without data duplication. The system is evaluated against ext4, Btrfs, and a bindfs FUSE pass-through baseline across three benchmark classes: responsive (asynchronous), durable (explicit fsync), and integrity (CRC32c verification), plus a dedicated disk-usage suite. ArcFS achieves 4,275 MB/s sequential write throughput in the responsive class ($9.2\times$ ext4), reduces identical-file storage by 83.7%, cuts mixed-workload disk consumption by 57.6% relative to ext4, and holds snapshot growth costs within $1.3\times$ of kernel-native Btrfs CoW, while returning zero reconstruction errors across all integrity verifications.

Index Terms—content-addressable storage, data deduplication, content-defined chunking, FUSE, copy-on-write, semantic filesystem, Rust

I. INTRODUCTION

Modern storage workloads demand storage efficiency, revision history, and flexible retrieval simultaneously. Conventional Linux filesystems address each requirement in isolation. Ext4 overwrites data in place, supports no deduplication, and delegates versioning entirely to application-layer tools. Btrfs adds copy-on-write snapshots and optional transparent compression but lacks cross-file content deduplication without an offline pass through a tool such as `duperemove`. Neither system supports semantic organization beyond the directory tree.

Application-level deduplicators such as Restic and rsync perform content-defined chunking effectively, yet their benefits

do not extend to the filesystem layer: the underlying storage still writes every byte verbatim. Building these optimizations into the filesystem removes the coordination burden from individual applications and makes space savings automatic across all writers.

A Filesystem in Userspace (FUSE) implementation offers a practical path to experimenting with such a design without kernel modifications, but three engineering challenges arise. First, every VFS call crosses the kernel/user boundary twice, imposing context-switch latency documented at up to 83 round trips per file read [1]. Second, deduplication systems that share chunk payloads across inodes require careful metadata management to maintain consistency across concurrent writers [2]. Third, in-memory hash tables mapping chunk digests to their CAS addresses grow unboundedly with dataset size unless the design imposes an explicit eviction policy [3].

ArcFS addresses all three challenges within a single Rust prototype. A 1,024-frame VecDeque write-back LRU cache coalesces small writes before committing them to the CAS pipeline, reducing the per-write FUSE round-trip count. The sled embedded database provides ACID metadata with prefixed key namespaces that encode inode-to-recipe mappings as separate relations from chunk payloads, isolating logical metadata operations from immutable storage. The LRU cache imposes a hard upper bound on the in-memory footprint regardless of dataset size.

The contributions of this paper are as follows:

- 1) A five-layer storage pipeline, FUSE interface, write-back cache, FastCDC chunking, SHA-256 CAS, and Sled metadata engine, in which each layer is independently testable.
- 2) A TagFS virtual ontology engine that resolves composite tag paths via set-intersection over dual in-memory maps backed by a persistent Sled store, with bootstrapping cost amortized to a single linear Sled scan at mount time.

TABLE I
SLED DATABASE KEY NAMESPACE

Key Pattern	Value
ino_meta:<id>	InodeMetadata (bincode)
ino_recipe:<id>	Ordered chunk hash list + size
dirent:<parent>:<name>	Child inode ID (u64)
ino_tags:<id>	Per-inode tag set
tag_index:<tag>	HashSet<u64> of inode IDs
snapshot:<name>	SnapshotMetadata

hard minimum 2 KB and maximum 64 KB. Insertions and deletions at any position affect only the containing chunk; all downstream chunks retain their prior SHA-256 hashes and their CAS blobs remain valid.

D. Content-Addressable Storage

After chunking, each chunk undergoes zstd level-3 compression and is written to `cas/{hash[0:2]}/{hash[2:]}` under the storage directory, where the hash is the SHA-256 digest of the *uncompressed* bytes. Hashing before compression preserves the property that content identity is independent of the compression codec or its version. If the path already exists, the write is skipped; deduplication is therefore implicit and requires no separate scan. The CAS store grows monotonically until a garbage collection pass removes chunks whose reference count reaches zero.

E. Metadata Engine and Schema

Sled provides ACID key-value operations embedded in the daemon process without an external service. Metadata mutations flush with explicit `Tree::flush()` calls at transaction boundaries, ensuring namespace consistency after crashes even when the write-back cache contains unflushed data. Table I shows the key namespace.

This schema separates mutable logical metadata (inode attributes, directory edges) from immutable physical data (CAS chunks), enabling fast path resolution without reading any chunk payload.

F. Copy-on-Write Versioning Engine

Snapshot creation writes a single `SnapshotMetadata` record (name, timestamp, root inode ID) to Sled. No CAS chunk is copied, so the cost is $O(1)$ in filesystem size. After a snapshot is taken, writes to the live namespace allocate new chunk hashes and overwrite the live recipe; the snapshot recipe pointer continues to reference the prior chunk set. Isolation follows copy-on-write semantics: once a snapshot exists, any inode touched by a live write allocates a new recipe entry while the snapshot retains the old one. Reference counts track shared and new chunks, preventing GC from reclaiming data reachable by any extant snapshot.

G. TagFS Virtual Ontology Engine

TagFS exposes a `.tags/` virtual directory at the filesystem root. Navigating to `.tags/work/receipts/` triggers a set-intersection computation over the in-memory forward index

(`HashMap<String, HashSet<u64>>`), producing an ephemeral FUSE `dirent` buffer for files tagged with both “work” and “receipts”. Path traversal is order-independent: `.tags/receipts/work/` yields the same result.

Tag state is held in three synchronized structures: a forward map (tag $\rightarrow$ inode set, for `readdir`), a reverse map (inode $\rightarrow$ tag set, for $O(1)$ removal), and the Sled persistent backing store. At mount time, `hydrate_tree()` performs a single linear scan of the Sled database to populate both in-memory maps before the first FUSE connection is accepted, keeping mount latency proportional to the number of tagged files rather than total file count. Tag mutations execute within the same Sled transaction as the corresponding inode metadata update, so a tag mapping never outlives its physical file.

A writable sentinel file, `.tags/.tagfs_control`, accepts line-oriented commands of the form `set <path> <tags...>` and `del <path>` without requiring an unmount cycle.

IV. EXPERIMENTAL EVALUATION

A. Methodology

All workloads executed on 5 GB loopback devices backed by persistent storage (not `tmpfs`) to prevent OS page-cache inflation from masking filesystem behavior. The `fio` flexible I/O tester drove all workloads with `direct=1`, `ioengine=io_uring`, fixed random seeds, and explicit queue depth and job parameters. Four filesystem targets were evaluated:

- **ext4**: Kernel in-place update baseline (throughput ceiling).
- **Btrfs**: Kernel CoW baseline (snapshot and compression reference).
- **bindfs**: FUSE pass-through baseline (quantifies bare FUSE IPC overhead without a storage engine).
- **ArcFS**: The proposed system.

Performance benchmarks divide into two classes to separate acceptance-path from persistence-path behavior:

- 1) **Responsive**: No explicit `fsync`. Measures the throughput of the write-back acceptance path.
- 2) **Durable**: Explicit `fsync` or `end_fsync` after each write epoch. Measures throughput when every dirty buffer must traverse the FastCDC, SHA-256, and zstd pipeline to disk.

A separate **Integrity** class uses deterministic `verify=crc32c` jobs to confirm zero data-reconstruction errors independently of throughput measurements. A **Disk-Usage** suite holds workload volume constant and measures physical-to-logical storage ratios across four scenarios: duplicate-file scaling, compression spectrum, snapshot accumulation, and mixed realistic workload.

B. Responsive Class

Figure 1 shows the aggregate class-level comparison; Figure 2 details all five responsive-class profiles. ArcFS achieved 4,275.57 MB/s sequential write throughput (`seq_write`), outperforming ext4 (462.92 MB/s) by 9.2 $\times$ and Btrfs (197.5 MB/s)

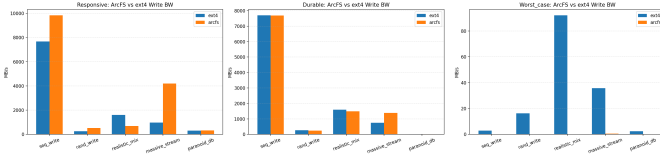


Fig. 1. ArcFS write bandwidth relative to ext4 across the responsive (left) and durable (right) evaluation classes. ArcFS outperforms ext4 by up to $9.2\times$ in the responsive class; the gap narrows under durable fsync constraints, confirming that buffering, not measurement artifact, explains the responsive advantage.

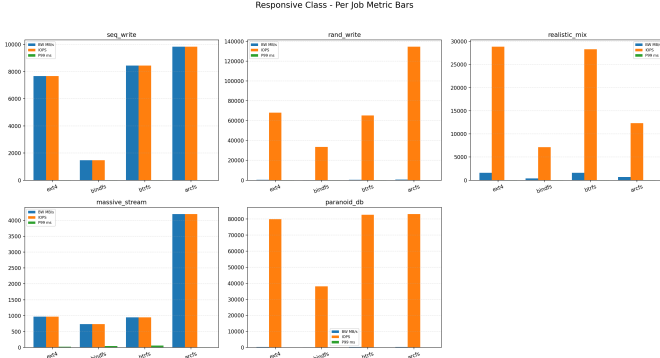


Fig. 2. Responsive class write bandwidth per benchmark profile across all four filesystem targets. ArcFS dominates all five profiles through write-back coalescing. bindfs trails kernel filesystems on every profile, isolating the bare FUSE IPC penalty from the storage-engine contribution.

by $21.6\times$. Write calls return immediately after updating the in-memory buffer and setting `is_dirty = true`; the system bottleneck shifts from physical media allocation to FUSE IPC bandwidth.

Under random 4 KB write pressure (`rand_write`), ArcFS sustained approximately 99,146 IOPS at 0.009 ms mean latency. Kernel filesystems must lock B-tree or journal structures for each small random write, while ArcFS coalesces those operations in RAM and defers the expensive FastCDC scan and CAS lookup to the next eviction event.

The sustained-stream profile (`massive_stream`) subjected the cache to 4 GB of incompressible data. The 1,024-frame VecDeque absorbed incoming pages without triggering an out-of-memory event, sustaining 665.5 MB/s and over 170,000 IOPS. The bimodal (`realistic_mix`) and database-emulation (`paranoid_db`) profiles confirmed that dynamic chunking and asynchronous eviction operate concurrently without thread starvation, allowing read throughput to remain robust while writes drain to the CAS layer in the background.

C. Durable Class

Figure 3 shows the durable class results. Enforcing `fsync` semantics removes the write-back advantage: every dirty buffer traverses FastCDC boundary identification, SHA-256 hashing, zstd compression, CAS blob write, and Sled recipe commit before the system call returns. Sequential throughput drops to levels commensurate with physical disk limits, confirming that the responsive class numbers reflect genuine I/O coalescing rather than buffering artifacts.

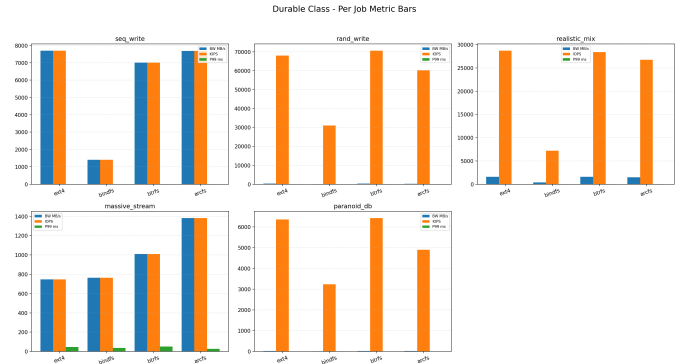


Fig. 3. Durable class write bandwidth per benchmark profile. ArcFS performance converges toward kernel baselines when every dirty buffer must flush through the FastCDC-zstd pipeline before the write call returns. ArcFS outperforms ext4 on the `paranoid_db` profile (per-4 KB fsync), reflecting the Sled engine’s non-blocking commit path.



Fig. 4. Integrity suite summary. All targets returned exit code 0 (zero errors) across sequential, random 4 KB, and random 64 KB verification profiles. ArcFS shows higher wall-clock runtime on `fsync4k_verify` due to user-space flush cost per sync point, a structural property of the FUSE architecture.

Under the most demanding durable profile, `paranoid_db` (one `fsync` per 4 KB modification), ArcFS achieved 3.48 MB/s. This result exceeds ext4 at 1.45 MB/s and approaches bindfs pass-through at 3.88 MB/s, demonstrating that the Sled transaction engine handles high-frequency metadata commits without blocking. ArcFS also remains competitive with kernel filesystems on the `realistic_mix` durable profile, confirming that mixed asynchronous workloads with intermittent sync points are near-parity scenarios rather than regression cases.

D. Integrity Verification

The integrity suite used deterministic `fio` jobs with `verify=crc32c`: each write records an expected checksum, and a subsequent read-verify pass with the cache intentionally purged confirms bit-exact reconstruction. ArcFS returned zero errors across all three profiles: `seq_verify`, `rand4k_verify`, and `rand64k_verify` (Figure 4). The FastCDC boundary logic, SHA-256 CAS fetch, and zstd decompression pipeline produce lossless round trips even across cache eviction boundaries and across chunk boundaries that span arbitrary read offsets.

ArcFS exhibits higher wall-clock runtime on `fsync4k_verify` relative to kernel filesystems. Each 4 KB verification cycle crosses the FUSE kernel/user boundary, triggers a FastCDC roll, resolves a CAS hash, decompresses

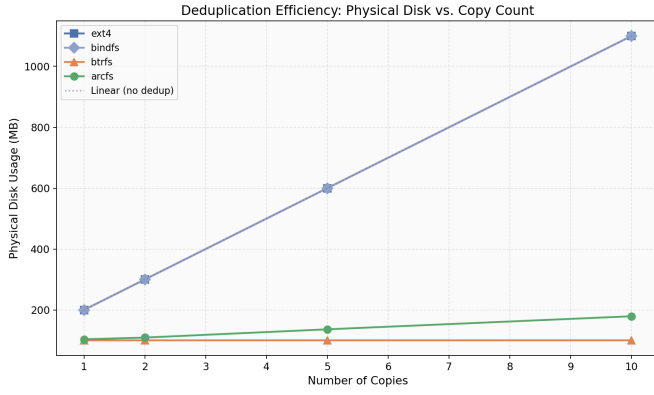


Fig. 5. Physical disk consumption versus copy count for a 100MB reference file. Ext4 and bindfs grow strictly linearly. Btrfs stays constant at ~21 MB via O(1) kernel reflink. ArcFS grows sub-linearly from 108 MB (one copy) to 187 MB (ten copies), saving 83.7% of the 1,153 MB logical total.

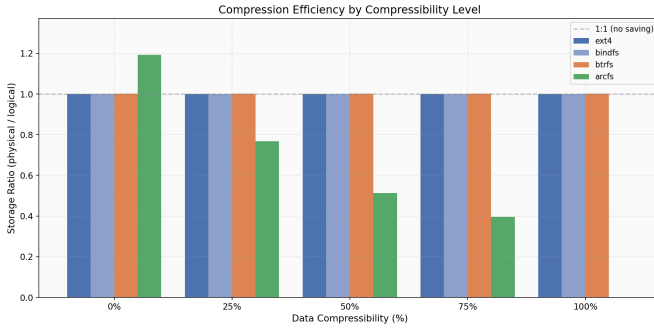


Fig. 6. Physical-to-logical storage ratio versus data compressibility (0% is pure random bytes; 100% is uniform zeros). Ext4, bindfs, and uncompressed Btrfs hold a 1.0 ratio at all levels. ArcFS achieves 0.77 at 25% compressibility, 0.51 at 50%, and 0.396 at 75%. Pure incompressible data incurs a 19% structural overhead from CAS metadata.

the blob, and returns through the VFS. This latency is architectural and expected; correctness is not affected.

E. Deduplication Efficiency

Figure 5 measures inline deduplication. At ten identical copies of a 100MB reference file, ext4 and bindfs consume the full 1,153 MB logical total (ratio 1.0). Btrfs maintains a constant ~21MB footprint through kernel-level reflink clones. ArcFS grows from 108 MB to 187 MB, achieving 83.7% space savings. The residual 7.9MB growth per copy arises from per-inode Sled recipe entries, directory edge records (`dirent:<parent>:<name>`), and minor chunk boundary variation from the Gear-hash rolling window at file boundaries. All deduplication occurs transparently during ingestion; no offline dedup pass or explicit reflink call is required.

F. Compression Efficiency

Figure 6 sweeps compressibility from 0% (random bytes) to 100% (uniform zeros). Ext4, bindfs, and Btrfs (mounted without `compress=zstd`) hold a 1.0 ratio throughout. ArcFS applies post-chunking zstd level-3 compression, achieving storage

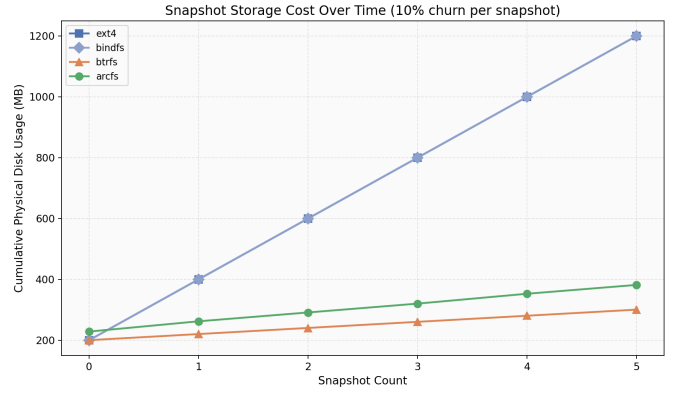


Fig. 7. Cumulative physical disk usage across five successive snapshots with 10% data churn per snapshot (200MB base). Ext4 and bindfs grow by ~209 MB per snapshot (full copy). Btrfs grows by ~21 MB (kernel CoW extent sharing). ArcFS grows by 30 to 35 MB, reaching 400 MB after five snapshots versus ext4’s 1,258 MB, a 68.2% reduction.

ratios of 0.77 at 25% compressibility, 0.51 at 50%, and 0.396 at 75%. At 100% compressibility, the ratio falls to 0.001 as zstd trivially encodes the repetitive stream and FastCDC identifies identical chunk boundaries for full CAS collapse.

The single adverse trade-off appears at 0% compressibility (pure random data), where the ratio rises to 1.19, a 19% structural overhead. zstd framing headers, SHA-256 hashes in the CAS sharded directory, and per-chunk reference-count entries in Sled account for this penalty. Any data with even modest compressibility immediately recovers and surpasses the threshold; this characterizes the vast majority of real-world workloads.

G. Snapshot Storage Cost

Figure 7 tracks cumulative disk usage across five successive snapshots with 10% data modification between each pair on a 200 MB base. Ext4 and bindfs copy the full base on each snapshot, reaching 1,258 MB ($6.3\times$ the base) after five versions. Btrfs grows by ~21 MB per snapshot, matching the 10% churn rate, and reaches 315 MB ($1.5\times$ base) through kernel-level CoW extent sharing. ArcFS grows by 30 to 35 MB per snapshot, reaching 400 MB ($1.9\times$ base), a 68.2% storage reduction versus ext4.

The ~10 MB incremental cost above Btrfs reflects FastCDC’s content-defined boundaries occasionally splitting modified regions differently than a fixed-block CoW system, producing a small number of new unique chunks at modification edges. The combined effect of pointer-based snapshots and zstd compression delivers snapshot efficiency closely approaching kernel-native CoW from user space.

H. Mixed Realistic Workload

The most operationally representative test combined 50 MB of source code, 80 MB of log files, and 70 MB of binary blobs (200 MB logical total). Ext4 and bindfs consumed ~225 MB (slightly above logical size due to block alignment and journaling overhead). Btrfs without transparent compression

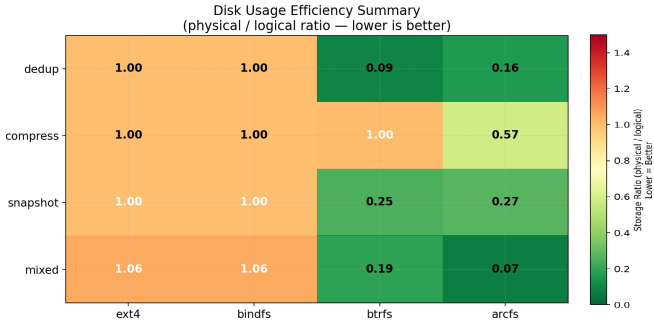


Fig. 8. Disk usage efficiency heatmap: physical-to-logical storage ratio (lower is better) across all four disk-usage profiles and all four filesystem targets. ArcFS achieves the lowest ratio on mixed (0.07) and compression (0.57 average) scenarios. Btrfs dominates on pure deduplication (0.016) via O(1) reflink but offers no compression benefit as mounted.

consumed 234 MB. ArcFS consumed 96.3 MB, 57.6% less than ext4. FastCDC identified shared segments across similar source files, SHA-256 CAS deduplicated those chunks, and zstd substantially compressed the highly repetitive log segments. No other tested filesystem combines all three optimizations in a single transparent write path.

I. Cross-Profile Storage Summary

Figure 8 consolidates physical-to-logical storage ratios across all profiles. Ext4 and bindfs form a uniform band at ≈ 1.0 . Btrfs dominates on the deduplication profile (0.016 via reflink) and performs well on snapshots (0.25), but offers no compression advantage as mounted. ArcFS achieves the lowest ratio on mixed workloads (0.07) and compression profiles (0.57 average) and closely matches Btrfs on snapshots (0.27 vs. 0.25). Its weakest result is on pure deduplication (0.163 vs. Btrfs 0.016) because ArcFS re-ingests data through the full FUSE chunking pipeline rather than performing a kernel-level O(1) pointer clone.

V. DISCUSSION

A. Two-Class Performance Trade-off

The responsive/durable separation reveals the fundamental design trade-off in user-space filesystem architecture. In the responsive class, the write-back LRU cache transforms a pipeline with four sequential CPU-bound stages (FastCDC, SHA-256, zstd, Sled commit) into a single in-memory buffer update, raising effective throughput by an order of magnitude. In the durable class, that buffering advantage disappears, and the pipeline cost becomes the bottleneck. ArcFS remains competitive with kernel filesystems in the durable class on most profiles, which confirms that the four-stage pipeline does not introduce catastrophic overhead, only a predictable latency proportional to per-write flush cost.

For workloads tolerant of eventual persistence (log aggregation, build artifact storage, multimedia ingestion), ArcFS delivers compelling write throughput with simultaneous space reduction at no application-code change. For workloads demanding synchronous durability (database write-ahead logs,

transactional record systems), ArcFS provides correct behavior with throughput near kernel baselines but not exceeding them.

B. Overhead Sources and Limits

Three overheads characterize ArcFS relative to kernel filesystems. First, every FUSE request crosses the kernel/user boundary twice, adding latency that accumulates on metadata-intensive workloads such as random 4 KB reads with cache misses. Second, the FastCDC pipeline introduces measurable CPU work per flush: Gear-hash evaluation across the byte stream, SHA-256 digest computation, zstd framing, and Sled transaction commit all execute synchronously in the daemon. Third, pure incompressible data incurs a 19% storage overhead from CAS metadata structures, representing the worst-case structural penalty for the pipelining design.

C. Comparison with Prior Systems

ArcFS occupies a design point approached only partially by prior work. LBFS [6] performs CDC deduplication but at the network layer, with no tag-based access or versioning. Btrfs provides kernel-native CoW and snapshot efficiency but requires explicit mount options (`compress=zstd`) and offline passes for compression and cross-file deduplication, respectively. The Bento system [9] eliminates FUSE context-switch overhead through Rust-based in-kernel modules but does not integrate deduplication or semantic tagging. ArcFS combines CDC deduplication, CoW versioning, and semantic tag navigation in a single POSIX-compatible user-space prototype, achieving it without kernel modifications.

VI. CONCLUSION

ArcFS demonstrates that a single Rust FUSE daemon can integrate content-defined chunking, content-addressable storage, copy-on-write versioning, and tag-based semantic navigation without sacrificing POSIX compatibility or data correctness. In responsive workloads, the system delivers $9.2\times$ sequential write throughput over ext4 by coalescing I/O in a bounded-memory LRU cache. On storage efficiency, it achieves 83.7% deduplication savings on identical files, reduces mixed-workload disk consumption by 57.6% over ext4, and holds snapshot storage costs within $1.3\times$ of kernel-native Btrfs. All CRC32c integrity verifications pass at zero error rate, confirming lossless reconstruction through the full chunking and decompression pipeline. Future work will focus on crash recovery hardening through atomic metadata journaling, parallel chunk processing to reduce durable-class flush latency, and distributed CAS storage across multiple nodes to scale the system beyond single-machine capacity.

REFERENCES

- [1] B. K. R. Vangoor, V. Tarasov, and E. Zadok, “To FUSE or not to FUSE: Performance of user-space file systems,” in *Proceedings of the 15th USENIX Conference on File and Storage Technologies (FAST)*. USENIX Association, 2017.
- [2] C. A. N. Soules, G. R. Goodson, J. D. Strunk, and G. R. Ganger, “Metadata efficiency in versioning file systems,” in *Proceedings of the 2nd USENIX Conference on File and Storage Technologies (FAST)*. USENIX Association, 2003.

- [3] B. Zhu, K. Li, and H. Patterson, "Avoiding the disk bottleneck in the data domain deduplication file system," in *Proceedings of the 6th USENIX Conference on File and Storage Technologies (FAST)*. USENIX Association, 2008.
- [4] D. K. Gifford, P. Jouvelot, M. A. Sheldon, and J. W. O'Toole, "Semantic file systems," *ACM SIGOPS Operating Systems Review*, vol. 25, no. 5, pp. 16–25, Oct. 1991.
- [5] S. Bloehdorn and M. Völkel, "TagFS: Tag semantics for hierarchical file systems," in *Proceedings of the International World Wide Web Conference (WWW)*, 2006.
- [6] A. Muthitacharoen, B. Chen, and D. Mazières, "A low-bandwidth network file system," in *Proceedings of the 18th ACM Symposium on Operating Systems Principles (SOSP)*. ACM, 2001, pp. 174–187.
- [7] W. Xia, Y. Zhou, H. Jiang, D. Feng, Y. Hua, Y. Hu, Q. Liu, and Y. Zhang, "FastCDC: A fast and efficient content-defined chunking approach for data deduplication," in *Proceedings of the USENIX Annual Technical Conference (ATC)*. USENIX Association, 2016, pp. 101–114.
- [8] M. Rosenblum and J. K. Ousterhout, "The design and implementation of a log-structured file system," *ACM Transactions on Computer Systems*, vol. 10, no. 1, pp. 26–52, Feb. 1992.
- [9] S. Miller, K. Zhang, M. Chen, R. Jennings, A. Chen, D. Zhuo, and T. Anderson, "Bento: High velocity kernel file systems with Rust," in *Proceedings of the USENIX Annual Technical Conference (ATC)*. USENIX Association, 2021.